

CSA1712

ARTIFICIAL INTELLIGENCE

NAME: VARSHINI M

REG NO.: 192511215

1. Solve the Water Jug Problem

Problem Statement

Two jugs are available:

- Jug A = **4 gallons**
- Jug B = **3 gallons**

Initially both are empty. The available operations are:

- Fill a jug completely.
- Empty a jug completely.
- Pour water from one jug into another until one jug is either full or empty.

Goal: Obtain exactly **2 gallons** in the 4-gallon jug.

State Representation

Represent each state as:

(A, B)

where

- A = amount of water in 4-gallon jug
- B = amount of water in 3-gallon jug

Initial State:

(0,0)

Goal State:

(2,x)

where x can be any value.

Solution Steps

Step	Action	State (4G,3G)
1	Fill 3-gallon jug	(0,3)
2	Pour into 4-gallon jug	(3,0)
3	Fill 3-gallon jug again	(3,3)
4	Pour into 4-gallon until full	(4,2)
5	Empty 4-gallon jug	(0,2)
6	Pour remaining 2 gallons into 4-gallon jug	(2,0)

Goal Achieved

The 4-gallon jug now contains exactly **2 gallons**.

Final State:

(2,0)

State Space Diagram

(0,0)

|

Fill 3G

|

(0,3)

|

Pour to 4G

|

(3,0)

|

Fill 3G

|

(3,3)

|

Pour to 4G

|

(4,2)

|

Empty 4G

|

(0,2)

|

Pour to 4G

|

(2,0) ← Goal

Interpretation

The Water Jug Problem is a classical **State Space Search** problem in Artificial Intelligence. It demonstrates how a sequence of legal actions can transform an initial state into a desired goal state.

2. Mars Rover Agent Analysis

i. Percepts

The Mars Rover receives information from:

- Camera images
- Rock composition sensors
- Temperature sensor
- Atmospheric pressure sensor
- Radiation detector
- Soil moisture sensor
- Wheel encoder
- GPS/Inertial Navigation System
- Obstacle detection sensors
- Battery level monitor

ii. Operating Environment

Observable

Partially Observable (cannot observe every part of Mars).

Agents

Single Agent.

Deterministic/Stochastic

Stochastic because terrain conditions are uncertain.

Episodic/Sequential

Sequential since current decisions affect future actions.

Static/Dynamic

Dynamic because environmental conditions change.

Discrete/Continuous

Continuous.

Known/Unknown

Partially Known.

iii. Actions

The Mars Rover can:

- Move forward
- Reverse
- Turn left
- Turn right
- Capture images
- Collect rock samples
- Drill surface
- Analyze soil
- Avoid obstacles
- Send data to Earth

- Recharge using solar panels
- Stop during dangerous conditions

iv. Performance Measure

Performance is evaluated by:

- Number of successful samples collected
- Mission completion rate
- Energy efficiency
- Navigation accuracy
- Obstacle avoidance
- Scientific discoveries
- Communication success
- Safety of rover

v. Suitable Agent Architecture

Utility-Based Learning Agent

Reason

A Mars Rover operates under uncertain conditions.

It must choose actions that maximize:

- Scientific value
- Battery life
- Safety
- Mission success

It continuously learns from sensor data and adapts to new terrain.

Therefore, a **Utility-Based Learning Agent** is the most suitable architecture.

Interpretation

The Mars Rover combines perception, reasoning, planning, and learning to autonomously explore Mars despite communication delays with Earth.

3. 8-Queens Problem

Problem Definition

Place eight queens on an **8×8 chessboard** such that:

- No two queens are in the same row.
- No two queens are in the same column.
- No two queens are on the same diagonal.

Problem Formulation

Initial State

Empty chessboard.

State Space

All possible placements of queens.

Total possible arrangements:

8^8

Operators

- Place a queen.
- Move a queen.
- Remove a queen.

Goal Test

All eight queens placed without conflicts.

Path Cost

Each move costs 1.

Valid Solution

Row : Column

1 → 1

2 → 5

3 → 8

4 → 6

5 → 3

6 → 7

7 → 2

8 → 4

Chessboard Representation

Q.....

....Q...

.....Q

.....Q..

..Q.....

.....Q.

.Q.....

...Q....

Search Strategy

The problem is commonly solved using:

- Backtracking Search
- Depth First Search (DFS)
- Hill Climbing
- Genetic Algorithm
- Simulated Annealing

Most efficient classical method:

Backtracking Search

Interpretation

The 8-Queens problem demonstrates how AI uses systematic search and constraint satisfaction to efficiently find a valid solution among millions of possibilities.

4. OLA Cab Booking Agent

Problem Statement

A customer wants to travel from one location to another using the OLA application.

The user selects cab type:

- Mini
- Sedan
- Shared
- Prime
- SUV

The system recommends the best option.

Suitable AI Agent

Utility-Based Agent

Reason

Different users have different preferences.

The agent evaluates:

- Cost
- Comfort
- Waiting time
- Distance
- Traffic
- Availability

- User ratings

It chooses the option with the highest utility.

PEAS Description

Performance Measure

- Minimum fare
- Minimum waiting time
- Comfortable ride
- Customer satisfaction
- Fast arrival

Environment

- Roads
- Maps
- Traffic
- Drivers
- Customers
- Weather

Actuators

- Display cab options
- Book ride
- Cancel ride
- Send notifications
- Track ride

Sensors

- GPS
- Traffic information
- Driver location

- Customer location
- Time
- Payment status

Algorithm (Pseudocode)

BEGIN

Input pickup location

Input destination

Display available cab types

Mini

Sedan

Shared

Prime

SUV

Calculate fare

Estimate waiting time

Check driver availability

FOR each cab

Calculate Utility Score

Utility =

Lowest Fare

+ Short Waiting Time

+ High Comfort

+ Driver Rating

END FOR

Select cab with Maximum Utility

Display Recommendation

Book Cab

END

Flow of the Agent

Customer Request

|



Get Pickup & Destination

|



Collect Traffic & Driver Data

|

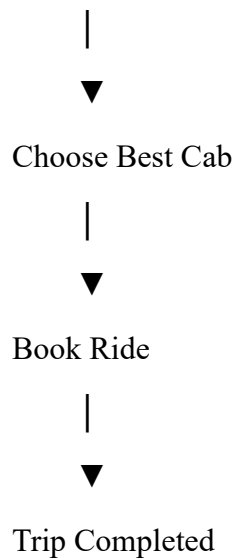


Calculate Fare

|



Evaluate Utility



5.

Problem Statement

A logistics company needs to transport packages from the **starting warehouse S** to the **destination warehouse G**. Each route has a transportation cost (fuel + time). The objective is to determine the **least-cost path** using the **Uniform Cost Search (UCS)** algorithm.

Given Weighted Graph

The routes and costs are:

Route Cost

S → A 1

S → G 12

A → B 3

A → C 1

B → D 3

C → D 1

C → G 2

D → G 3

Algorithm Used

Uniform Cost Search (UCS)

Uniform Cost Search expands the node with the **lowest cumulative path cost** first. It guarantees the **optimal (least-cost) path** when all edge costs are positive.

State Representation

Each state is represented as:

(Current Node, Total Cost)

Initial State

(S, 0)

Goal State

(G, Minimum Cost)

Step-by-Step UCS Execution

Step 1

Start at S

Priority Queue:

S (0)

Expand S

Add neighbors:

- A (1)
- G (12)

Queue:

A (1)

G (12)

Step 2

Expand A (1)

Neighbors:

- $B = 1 + 3 = 4$
- $C = 1 + 1 = 2$

Queue:

C (2)

B (4)

G (12)

Step 3

Expand **C (2)**

Neighbors:

- $D = 2 + 1 = 3$
- $G = 2 + 2 = 4$

Queue:

D (3)

B (4)

G (4)

G (12)

Step 4

Expand **D (3)**

Neighbor:

- $G = 3 + 3 = 6$

Queue:

B (4)

G (4)

G (6)

G (12)

Step 5

Expand **B (4)**

Neighbor:

- $D = 4 + 3 = 7$

Existing path to D is cost **3**, so ignore this higher-cost path.

Queue:

G (4)

G (6)

G (12)

Step 6

Expand **G (4)**

Goal node reached.

Since UCS always selects the node with the **lowest cumulative cost**, this is the **optimal solution**.

Optimal Path

$S \rightarrow A \rightarrow C \rightarrow G$

Total Cost Calculation

$S \rightarrow A = 1$

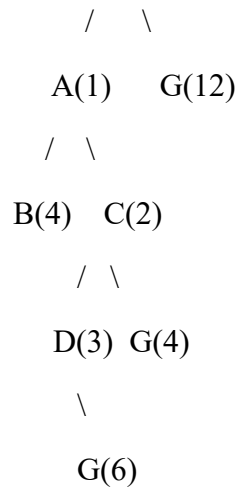
$A \rightarrow C = 1$

$C \rightarrow G = 2$

Total Cost = 4

Search Tree

S(0)



Pseudocode

BEGIN

Insert Start node S into Priority Queue

Cost(S) = 0

WHILE Priority Queue is not empty

Remove node with minimum cost

IF node is Goal (G)

RETURN path and total cost

Expand current node

FOR each neighboring node

Calculate new cumulative cost

IF new cost is lower than previous cost

Update cost

Insert into Priority Queue

END WHILE

END

Result

- **Algorithm Used:** Uniform Cost Search (UCS)
- **Optimal Path:** $S \rightarrow A \rightarrow C \rightarrow G$
- **Minimum Total Cost:** 4